

Machine Learning Algorithms Laboratory

Experiment 6

Dimensionality Reduction and Model Evaluation Using PCA

Experiment No.	6
Experiment Title	Dimensionality Reduction and Model Evaluation Using PCA
Student Name	Harshita B
Register Number	3122247001021
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	31st August 2026
GitHub Repository	https://github.com/Harshitaab/ML_assignments_3122247001021

1 Objective

The objective of this experiment is to study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the performance of different machine-learning classifiers. Two experimental settings are compared: (i) models trained using the original standardized feature space without PCA, and (ii) models trained using a PCA-reduced feature space. Hyperparameter tuning using GridSearchCV and stratified five-fold cross-validation are employed to make the comparison systematic. The experiment evaluates SVM, Naïve Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and a Stacking ensemble.

2 Problem Statement

The problem is to determine whether reducing the dimensionality of the feature space improves or degrades classification performance and validation stability. The input consists of numerical measurements from the Breast Cancer Wisconsin Diagnostic dataset, while the output is the predicted binary class. The experiment first establishes a baseline using the original 30 features and then transforms the standardized training data into principal components. The reduced representation is selected to retain approximately 95% of the training variance. The resulting classifiers are compared using test-set accuracy, precision, recall, F1-score, five-fold cross-validation mean F1-score, and cross-validation standard deviation.

3 Dataset Description

The Breast Cancer Wisconsin Diagnostic dataset supplied by scikit-learn was used. It contains numerical measurements derived from digitized images of breast-mass cell nuclei. The target contains two classes, malignant and benign. The dataset contains 569 observations and 30 numerical features, with 212 malignant and 357 benign observations. No missing values were found in the predictor matrix. The data were divided using an 80:20 stratified train-test split, resulting in 455 training samples and 114 test samples.

Dataset Property	Value
Dataset Name	Breast Cancer Wisconsin Diagnostic dataset
Dataset Source	scikit-learn (<code>load_breast_cancer</code>)
Number of Samples	569
Number of Features	30 numerical features
Number of Classes	2
Classes	Malignant (212), Benign (357)
Missing Values	0
Train-Test Split	80% training (455), 20% testing (114)
Random State	42

4 Exploratory Data Analysis

4.1 Dataset Overview and Statistical Summary

The first observations and descriptive statistics were inspected before model training. All predictor variables are numerical. The missing-value check returned zero, so no imputation procedure was required. The descriptive statistics also showed that the numerical features have substantially different scales. This makes standardization particularly important because PCA is variance-based and distance- or margin-based algorithms such as KNN and SVM are sensitive to feature scale.

4.2 Class Distribution



Figure 1: Distribution of the malignant and benign target classes.

Inference: The dataset contains 212 malignant observations and 357 benign observations. Thus, the classes are not perfectly balanced, although both classes contain a sub-

stantial number of examples. Because accuracy alone can hide differences in class-wise performance, precision, recall, and F1-score are also reported. The stratified train-test split and StratifiedKFold procedure preserve approximately the same class proportions across evaluation subsets.

4.3 Exploratory Analysis Summary

The EDA confirms that the dataset is numerical, contains no missing predictor values, and has features on different scales. These characteristics make the dataset appropriate for demonstrating standardization followed by PCA. A separate correlation-matrix visualization was not generated in the executed notebook; therefore, no additional correlation figure is claimed as an experimental output.

5 Data Preprocessing

5.1 Train-Test Split

The data were split into 80% training and 20% testing subsets using `train_test_split` with `stratify=y` and `random_state=42`. The training set contains 455 observations and is used for hyperparameter tuning and cross-validation. The 114-sample test set is kept independent for the final evaluation.

5.2 Standardization

Each numerical feature was standardized using

$$z = \frac{x - \mu}{\sigma},$$

where x is the original feature value, μ is the feature mean, and σ is its standard deviation. Standardization prevents features with larger numerical scales from dominating PCA and also provides a consistent representation for the classifiers.

5.3 Pipeline and Data Leakage Prevention

For the No-PCA setting, the pipeline is

Original Features $\rightarrow$ StandardScaler $\rightarrow$ Classifier.

For the PCA setting, the pipeline is

Original Features $\rightarrow$ StandardScaler $\rightarrow$ PCA $\rightarrow$ Classifier.

The scaler and PCA transformation are fitted within the cross-validation pipeline. Consequently, information from a validation fold is not used to fit preprocessing transformations for that fold, reducing the risk of data leakage.

6 Principal Component Analysis

PCA transforms the original correlated feature space into a new orthogonal coordinate system called principal components. The first component captures the maximum possible variance; each subsequent component captures the maximum remaining variance subject to orthogonality with the preceding components.

If the standardized feature vector is $\mathbf{x}$ and the principal directions are stored in the matrix $\mathbf{W}$, the transformed representation can be expressed as

$$\mathbf{z} = \mathbf{W}^T \mathbf{x}.$$

The explained-variance ratio of component i describes the fraction of total standardized-data variance captured by that component.

6.1 Component Selection

The original feature space contains 30 features. The cumulative explained-variance analysis selected 10 principal components to retain at least 95% of the training variance. The actual retained cumulative variance was

$$95.2677\%.$$

Thus, PCA reduced the dimensionality from 30 to 10 features while retaining approximately 95% of the variance.

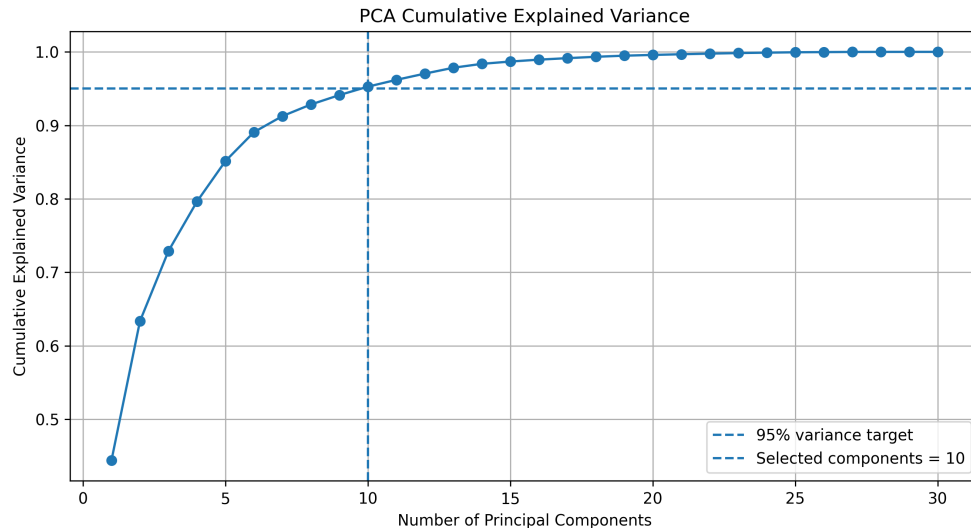


Figure 2: Cumulative explained variance as a function of the number of principal components.

Inference: The cumulative curve increases as additional principal components are included. The horizontal reference corresponds to the 95% variance target, and the selected point occurs at 10 components. Therefore, ten components are sufficient to retain at least the required variance. This provides an objective basis for selecting the PCA dimensionality instead of choosing the number of components arbitrarily.

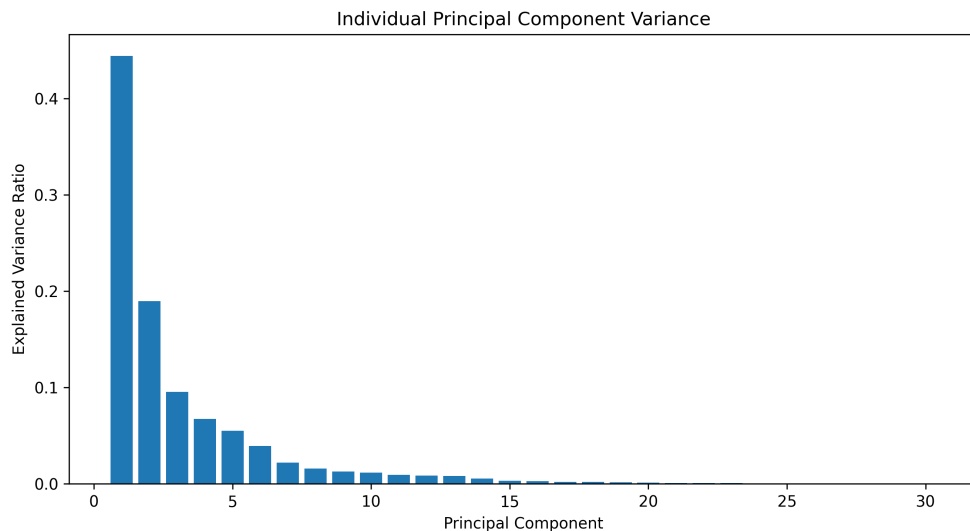


Figure 3: Explained variance ratio of the individual principal components.

Inference: The first few principal components capture substantially more variance than later components. The decreasing contribution of later components indicates redundancy in the original 30-dimensional representation. This demonstrates why a lower-dimensional representation can preserve much of the variation while removing several low-contribution directions.

7 Machine Learning Algorithms

7.1 Support Vector Machine

SVM seeks a decision boundary that maximizes the margin between classes. For a linear classifier, the decision function can be represented as

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b.$$

The parameter C controls the trade-off between maximizing the margin and penalizing classification errors. The RBF kernel can additionally represent nonlinear boundaries. SVM can be sensitive to feature scaling and the geometry of the feature space, making it relevant to a PCA comparison.

Advantages: effective in high-dimensional spaces, strong margin-based classification, and flexible kernel choices.

Limitations: sensitive to hyperparameter selection and can be computationally expensive for larger datasets.

7.2 Naïve Bayes

Gaussian Naïve Bayes applies Bayes' theorem while assuming conditional independence among features. For a feature vector $\mathbf{x}$ and class c ,

$$P(c \mid \mathbf{x}) \propto P(c) \prod_i P(x_i \mid c).$$

It is computationally efficient and works well as a probabilistic baseline.

Advantages: simple, fast, and effective for many classification problems.

Limitations: the conditional-independence assumption may not hold for correlated features.

7.3 K-Nearest Neighbors

KNN predicts the class of an observation from the labels of its nearest training examples. For distance-based KNN, feature geometry directly determines neighborhood membership. In this experiment, both Euclidean and Manhattan metrics were considered.

Advantages: simple, non-parametric, and capable of modelling nonlinear class boundaries.

Limitations: sensitive to scaling, the choice of k , and irrelevant or redundant dimensions.

7.4 Logistic Regression

Logistic Regression models the probability of the positive class using the sigmoid function:

$$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + \exp[-(\mathbf{w}^T \mathbf{x} + b)]}.$$

It provides a linear decision boundary in the transformed feature space.

Advantages: interpretable, computationally efficient, and effective for linearly separable or approximately linear problems.

Limitations: limited when complex nonlinear relationships are essential.

7.5 Decision Tree

A Decision Tree recursively partitions the feature space using feature thresholds. Splits are selected to increase class purity according to criteria such as Gini impurity or entropy.

Advantages: interpretable and capable of modelling nonlinear relationships without requiring feature scaling.

Limitations: individual trees can be unstable and prone to overfitting.

7.6 Random Forest

Random Forest combines multiple decision trees trained using randomized samples and feature subsets. The final prediction is obtained by aggregating the predictions of the trees.

Advantages: robust, nonlinear, and generally less prone to overfitting than a single decision tree.

Limitations: larger computational and memory requirements and reduced interpretability compared with one tree.

7.7 AdaBoost

AdaBoost constructs an ensemble sequentially, giving greater emphasis to observations that previous weak learners classified incorrectly. The final classifier combines the weak learners with weighted contributions.

Advantages: can improve weak learners substantially and model nonlinear decision boundaries.

Limitations: can be sensitive to noisy or difficult observations.

7.8 Gradient Boosting

Gradient Boosting builds an additive sequence of weak learners, typically shallow trees, where each new learner attempts to reduce the remaining loss.

Advantages: powerful nonlinear modelling and strong predictive performance.

Limitations: sensitive to hyperparameters and potentially computationally expensive.

7.9 XGBoost

XGBoost is an optimized gradient-boosting framework that builds an ensemble of trees using regularized objective optimization.

Advantages: strong predictive performance, regularization, and efficient tree boosting.

Limitations: hyperparameter tuning can be complex and the resulting model is less directly interpretable.

7.10 Stacking

Stacking combines predictions from multiple base learners and supplies them to a meta-learner. The implementation used SVM, KNN, and Random Forest as base learners and Logistic Regression as the final meta-learner. Stacking was evaluated separately using a fixed configuration rather than GridSearchCV.

8 Experimental Setup

Python Environment	Python-based Jupyter Notebook
Dataset Library	scikit-learn
Numerical Processing	NumPy, pandas
Visualization	Matplotlib
Machine Learning	scikit-learn, XGBoost
Validation	StratifiedKFold, 5 folds
Hyperparameter Tuning	GridSearchCV
Scoring for Selection	F1-score
Train-Test Split	80:20, stratified
Random State	42
PCA Target	At least 95% variance
Selected Components	10

9 Hyperparameter Selection

GridSearchCV was used for the nine individual classifiers. The parameter combinations were evaluated using stratified five-fold cross-validation and F1-score as the model-selection metric. The number of combinations tested was: SVM–12, Naïve Bayes–3, KNN–16, Logistic Regression–8, Decision Tree–12, Random Forest–12, AdaBoost–6, Gradient Boosting–8, and XGBoost–8.

Table 2: Best hyperparameters selected for each model and setting.

Model	Setting	Best Configuration	CV F1	Std.
SVM	No PCA	$C = 0.1$, kernel=linear, gamma=scale	0.9810	0.0082
SVM	With PCA	$C = 10$, kernel=linear, gamma=scale	0.9843	0.0065
Naive Bayes	No PCA	var_smoothing= 10^{-9}	0.9478	0.0228
Naive Bayes	With PCA	var_smoothing= 10^{-9}	0.9433	0.0134
KNN	No PCA	$k = 3$, weights=uniform, metric=manhattan	0.9774	0.0104
KNN	With PCA	$k = 7$, weights=uniform, metric=manhattan	0.9742	0.0152
Logistic Regression	No PCA	$C = 0.1$, solver=lbfgs	0.9861	0.0042
Logistic Regression	With PCA	$C = 1$, solver=lbfgs	0.9861	0.0042
Decision Tree	No PCA	max_depth=3, min_samples_split=5	0.9418	0.0034
Decision Tree	With PCA	max_depth=None, min_samples_split=10	0.9472	0.0292
Random Forest	No PCA	n_estimators=100, max_depth=None, min_samples_split=2	0.9699	0.0146
Random Forest	With PCA	n_estimators=100, max_depth=None, min_samples_split=5	0.9687	0.0179
AdaBoost	No PCA	n_estimators=100, learning_rate=1	0.9844	0.0150
AdaBoost	With PCA	n_estimators=50, learning_rate=1	0.9811	0.0136
Gradient Boosting	No PCA	n_estimators=100, learning_rate=0.1, max_depth=3	0.9614	0.0121
Gradient Boosting	With PCA	n_estimators=100, learning_rate=0.1, max_depth=3	0.9579	0.0203
XGBoost	No PCA	n_estimators=100, max_depth=3, learning_rate=0.1	0.9772	0.0143

XGBoost	With PCA	n_estimators=100, max_depth=3, learning_rate=0.1	0.9667	0.0103
Stacking	No PCA	Fixed configuration (SVM + KNN + Random Forest; Logistic Regression meta-learner)	0.9807	0.0089
Stacking	With PCA	Fixed configuration (SVM + KNN + Random Forest; Logistic Regression meta-learner)	0.9774	0.0089

Interpretation: The best configuration is selected using the mean validation F1-score rather than by manually choosing a single parameter setting. This makes the comparison more systematic. The selected parameters also differ between No-PCA and PCA settings for several classifiers, showing that the preferred model configuration can depend on the feature representation.

10 Results

10.1 Independent Test-Set Metrics

The following table reports the final independent test-set metrics. Precision, recall, and F1-score are computed for the positive class used by scikit-learn in the binary classification evaluation.

Table 3: Independent test-set performance for all models.

Model	Setting	Accuracy	Precision	Recall	F1
SVM	No PCA	0.9825	0.9861	0.9861	0.9861
SVM	With PCA	0.9649	0.9857	0.9583	0.9718
Naive Bayes	No PCA	0.9298	0.9444	0.9444	0.9444
Naive Bayes	With PCA	0.9211	0.9315	0.9444	0.9379
KNN	No PCA	0.9649	0.9595	0.9861	0.9726
KNN	With PCA	0.9561	0.9589	0.9722	0.9655
Logistic Regression	No PCA	0.9737	0.9726	0.9861	0.9793
Logistic Regression	With PCA	0.9737	0.9859	0.9722	0.9790
Decision Tree	No PCA	0.9386	0.9452	0.9583	0.9517
Decision Tree	With PCA	0.9035	0.9296	0.9167	0.9231
Random Forest	No PCA	0.9561	0.9589	0.9722	0.9655

Random Forest	With PCA	0.9298	0.9444	0.9444	0.9444
AdaBoost	No PCA	0.9561	0.9467	0.9861	0.9660
AdaBoost	With PCA	0.9474	0.9583	0.9583	0.9583
Gradient Boosting	No PCA	0.9561	0.9467	0.9861	0.9660
Gradient Boosting	With PCA	0.9561	0.9467	0.9861	0.9660
XGBoost	No PCA	0.9474	0.9459	0.9722	0.9589
XGBoost	With PCA	0.9474	0.9583	0.9583	0.9583
Stacking	No PCA	0.9737	0.9859	0.9722	0.9790
Stacking	With PCA	0.9561	0.9718	0.9583	0.9650

10.2 Five-Fold Cross-Validation

The training set was evaluated using five-fold stratified cross-validation. The mean F1-score indicates average validation performance, while the standard deviation measures fold-to-fold variability. Lower standard deviation indicates greater stability.

Table 4: Fold-wise F1-scores for the selected configurations.

Model	Setting	F1-1	F1-2	F1-3	F1-4	F1-5	Mean	Std.
SVM	No PCA	0.9661	0.9825	0.9825	0.9828	0.9913	0.9810	0.0082
SVM	With PCA	0.9739	0.9913	0.9913	0.9828	0.9825	0.9843	0.0065
Naive Bayes	No PCA	0.9310	0.9735	0.9123	0.9655	0.9565	0.9478	0.0228
Naive Bayes	With PCA	0.9244	0.9565	0.9310	0.9573	0.9474	0.9433	0.0134
KNN	No PCA	0.9828	0.9913	0.9649	0.9655	0.9825	0.9774	0.0104
KNN	With PCA	0.9828	0.9913	0.9649	0.9492	0.9828	0.9742	0.0152
Logistic Regression	No PCA	0.9828	0.9825	0.9913	0.9828	0.9913	0.9861	0.0042
Logistic Regression	With PCA	0.9828	0.9911	0.9913	0.9828	0.9825	0.9861	0.0042
Decision Tree	No PCA	0.9391	0.9455	0.9381	0.9402	0.9464	0.9418	0.0034

Decision Tree	With PCA	0.9180	0.9825	0.9310	0.9828	0.9217	0.9472	0.0292
Random Forest	No PCA	0.9735	0.9643	0.9464	0.9744	0.9911	0.9699	0.0146
Random Forest	With PCA	0.9483	0.9825	0.9474	0.9739	0.9913	0.9687	0.0179
AdaBoost	No PCA	0.9913	1.0000	0.9565	0.9828	0.9913	0.9844	0.0150
AdaBoost	With PCA	0.9661	1.0000	0.9825	0.9655	0.9913	0.9811	0.0136
Gradient Boosting	No PCA	0.9649	0.9558	0.9464	0.9573	0.9825	0.9614	0.0121
Gradient Boosting	With PCA	0.9310	0.9739	0.9369	0.9825	0.9649	0.9579	0.0203
XGBoost	No PCA	0.9913	0.9825	0.9558	0.9655	0.9911	0.9772	0.0143
XGBoost	With PCA	0.9573	0.9739	0.9550	0.9825	0.9649	0.9667	0.0103
Stacking	No PCA	0.9828	0.9825	0.9643	0.9828	0.9913	0.9807	0.0089
Stacking	With PCA	0.9744	0.9825	0.9649	0.9739	0.9913	0.9774	0.0089

Interpretation: Models with high mean F1 and low standard deviation provide both strong and consistent validation performance. The fold-wise results show that the effect of PCA is model-dependent. In particular, Naïve Bayes becomes substantially more stable after PCA, while Decision Tree exhibits a marked increase in fold variability.

10.3 Main No-PCA versus PCA Comparison

Table 5: Overall comparison of No-PCA and PCA configurations.

Model	Acc. NP	Acc. P	F1 NP	F1 P	$\Delta F1$	CV NP	CV P	Std NP	Std P
SVM	0.9825	0.9649	0.9861	0.9718	-0.0143	0.9810	0.9843	0.0082	0.0065
Naive Bayes	0.9298	0.9211	0.9444	0.9379	-0.0065	0.9478	0.9433	0.0228	0.0134
KNN	0.9649	0.9561	0.9726	0.9655	-0.0071	0.9774	0.9742	0.0104	0.0152
Logistic Regression	0.9737	0.9737	0.9793	0.9790	-0.0003	0.9861	0.9861	0.0042	0.0042
Decision Tree	0.9386	0.9035	0.9517	0.9231	-0.0286	0.9418	0.9472	0.0034	0.0292
Random Forest	0.9561	0.9298	0.9655	0.9444	-0.0211	0.9699	0.9687	0.0146	0.0179
AdaBoost	0.9561	0.9474	0.9660	0.9583	-0.0077	0.9844	0.9811	0.0150	0.0136
Gradient Boosting	0.9561	0.9561	0.9660	0.9660	+0.0000	0.9614	0.9579	0.0121	0.0203

XGBoost	0.9474	0.9474	0.9589	0.9583	-0.0006	0.9772	0.9667	0.0143	0.0103
Stacking	0.9737	0.9561	0.9790	0.9650	-0.0140	0.9807	0.9774	0.0089	0.0089

Key observations:

- PCA did not increase the independent test F1-score for any model. Gradient Boosting was unchanged at 0.9660, while all other models decreased.
- The largest test F1 decrease occurred for Decision Tree, falling from 0.9517 to 0.9231, a change of -0.0286.
- SVM showed the largest positive change in cross-validation mean F1, increasing from 0.9810 to 0.9843.
- Decision Tree also showed a small increase in CV mean F1, but its test performance fell substantially and its CV standard deviation increased sharply.
- Naïve Bayes experienced the largest reduction in CV standard deviation, from 0.0228 to 0.0134, indicating improved fold stability.
- Overall, PCA improved validation mean F1 for SVM and Decision Tree, but this did not translate into improved independent test-set F1 for either model.

11 Result Visualization

11.1 PCA Effect on F1-Score

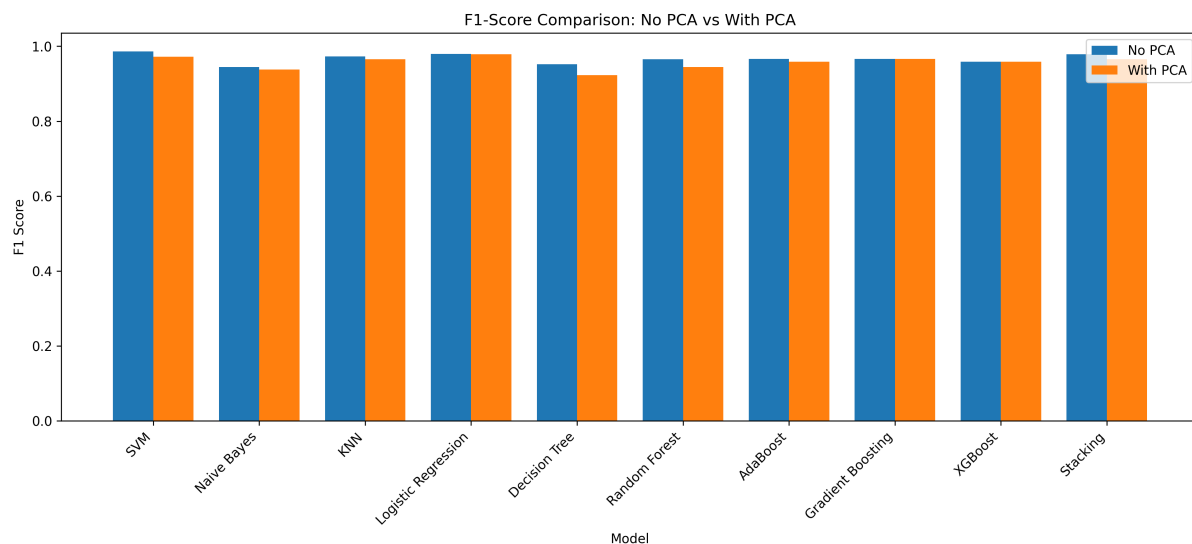


Figure 4: Test-set F1-score comparison between the original feature space and the PCA-reduced feature space.

Inference: The figure shows that the No-PCA configuration generally achieves equal or higher test F1-score. Gradient Boosting has essentially identical F1-score in both settings, whereas Decision Tree and Random Forest show noticeable reductions after PCA. Logistic Regression changes only marginally, indicating that it is comparatively insensitive to the dimensionality reduction for this dataset.

11.2 PCA Effect on Accuracy

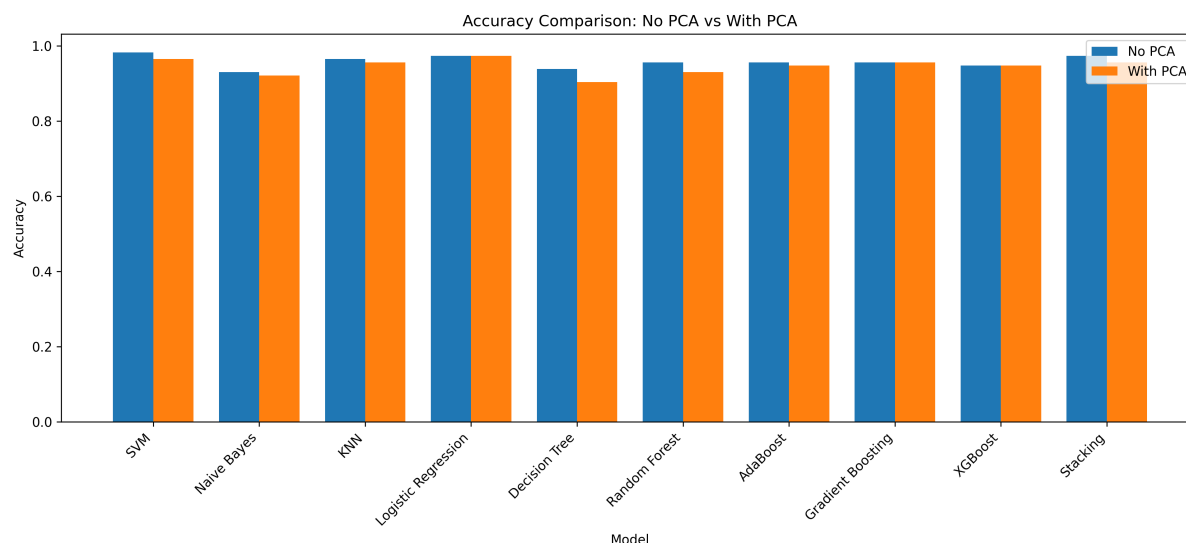


Figure 5: Test-set accuracy comparison between No PCA and With PCA.

Inference: The accuracy comparison shows that PCA either leaves performance unchanged or reduces it for the evaluated classifiers. Logistic Regression and Gradient Boosting retain the same accuracy in both settings. The largest accuracy decrease is observed for Decision Tree, from 0.9386 to 0.9035. This indicates that retaining 95% of variance does not necessarily preserve all information that is useful for a particular classifier.

11.3 Confusion Matrix of the Best PCA Model

The best PCA model among the individual classifiers and stacking ensemble is Logistic Regression based on test-set F1-score. Its confusion matrix is shown below.

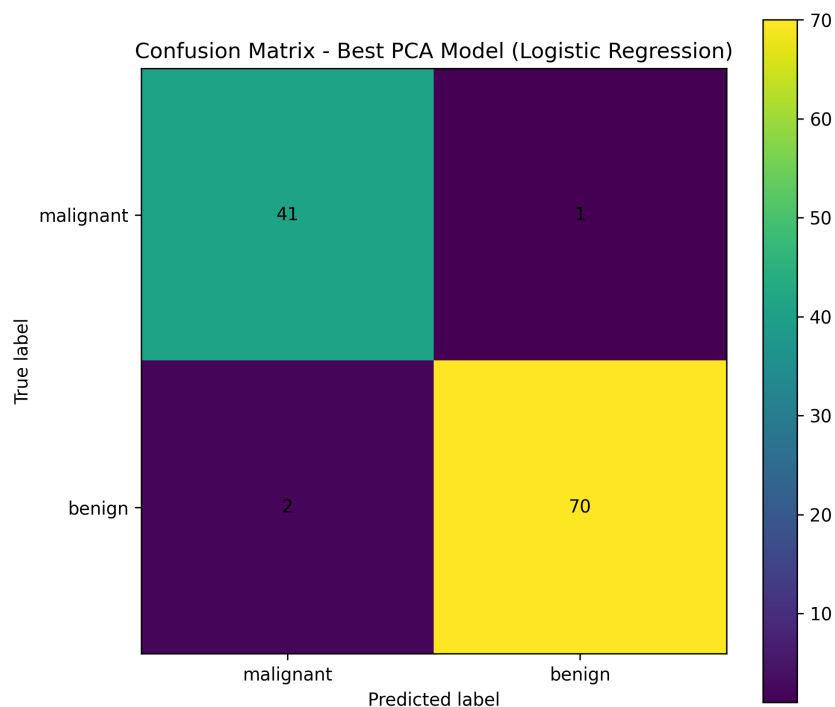


Figure 6: Confusion matrix for the best PCA model, Logistic Regression.

Inference: The confusion matrix contains 41 correctly classified malignant cases and 70 correctly classified benign cases. Only three test samples are misclassified: one malignant sample is predicted as benign and two benign samples are predicted as malignant. The small number of off-diagonal entries explains the high test accuracy of 0.9737 and F1-score of 0.9790. The class-wise report gives malignant precision 0.95 and recall 0.98, while benign precision is 0.99 and recall is 0.97.

11.4 ROC Curve

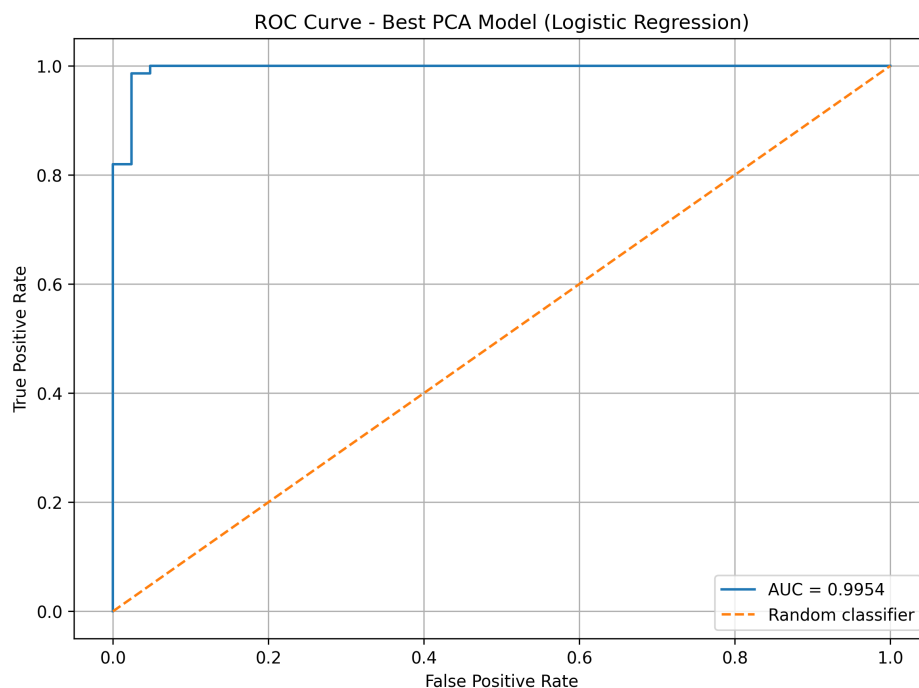


Figure 7: ROC curve for the best PCA Logistic Regression model.

Inference: The ROC curve lies substantially above the diagonal random-classifier line. The ROC-AUC is 0.9954, indicating excellent ranking-based separation of the two classes on the test set. The curve reaches a high true-positive rate at a relatively low false-positive rate. This complements the threshold-dependent accuracy and F1-score by evaluating discrimination across multiple thresholds.

11.5 Precision-Recall Curve

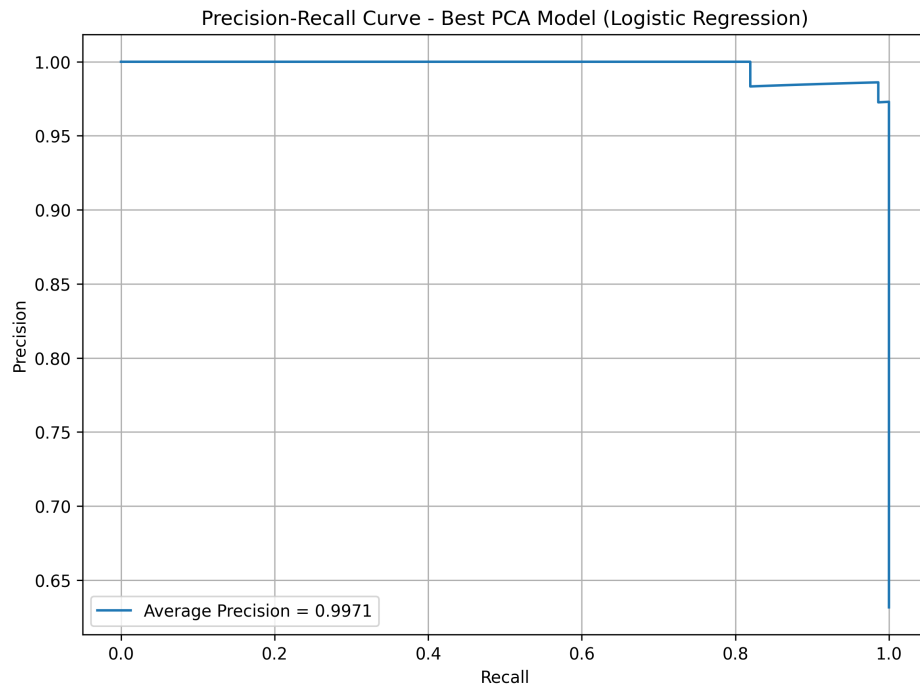


Figure 8: Precision-Recall curve for the best PCA Logistic Regression model.

Inference: The precision remains very high over most of the recall range, indicating that the model can identify positive samples while producing relatively few false-positive predictions. The Average Precision is 0.9971. The curve therefore supports the strong classification quality indicated by the confusion matrix and F1-score.

11.6 Cross-Validation Stability

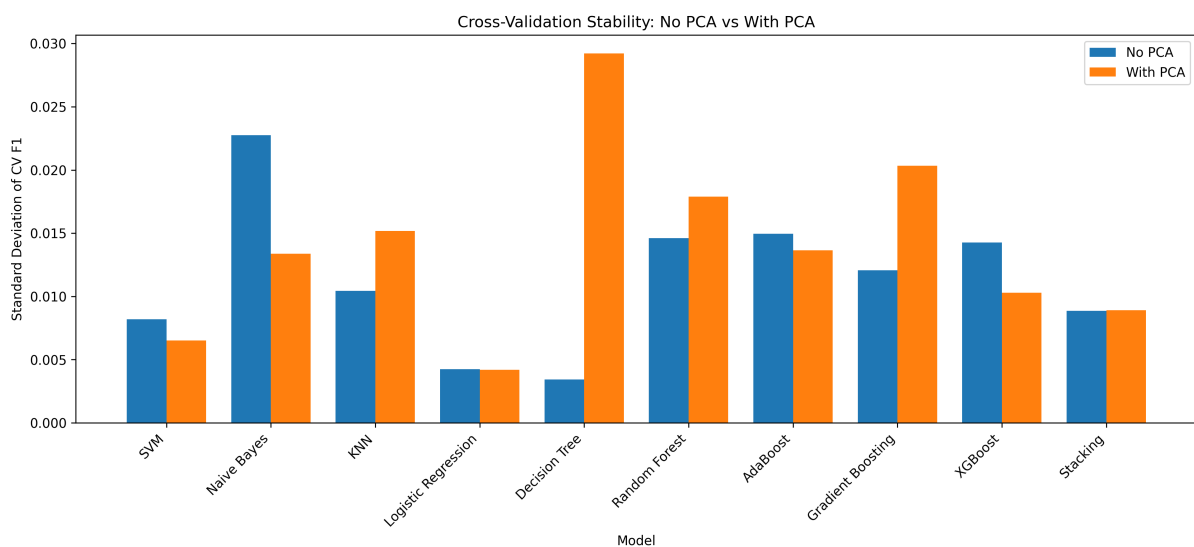


Figure 9: Five-fold cross-validation F1 standard deviation for No-PCA and PCA configurations.

Inference: Lower bars indicate more stable validation performance. PCA reduces the standard deviation for SVM, Naïve Bayes, and Logistic Regression, while increasing it for KNN, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, and XGBoost. The most substantial improvement in stability is observed for Naïve Bayes. The largest increase in variability occurs for Decision Tree, whose standard deviation rises from 0.0034 to 0.0292.

12 Discussion

12.1 Which Models Improved Most with PCA?

Based on independent test-set F1-score, no classifier improved after PCA. Gradient Boosting was the only model whose F1-score remained unchanged at 0.9660. Logistic Regression showed the smallest decrease, from 0.9793 to 0.9790. The largest decrease was observed for Decision Tree, whose F1-score fell by 0.0286.

The behaviour is consistent with the nature of the algorithms. SVM, KNN, and Logistic Regression depend more directly on the geometry of the feature space, so PCA can sometimes be beneficial when redundancy or correlation dominates. However, PCA creates linear combinations of the original variables. Tree-based models make threshold decisions on individual features, so replacing original variables with principal components can remove feature-specific information that is useful for their splits.

12.2 Did PCA Reduce Variance Across Folds?

PCA did not universally reduce validation variability. It reduced the standard deviation for:

- SVM: $0.0082 \rightarrow 0.0065$,
- Naïve Bayes: $0.0228 \rightarrow 0.0134$,
- Logistic Regression: $0.0042 \rightarrow 0.0042$ (small reduction).

For the remaining models, the standard deviation increased. The strongest stability improvement occurred for Naïve Bayes, while Decision Tree showed the largest increase in variability. Therefore, PCA cannot be described as a universally stabilizing transformation for this experiment.

12.3 Was PCA Beneficial for Reducing Overfitting?

PCA reduced the feature space from 30 dimensions to 10 dimensions, which can reduce model complexity and potentially suppress redundant or noisy directions. However, the experimental results do not provide evidence that PCA universally reduced overfitting. Several models had lower test F1 after PCA, even when their cross-validation mean F1 was similar or slightly higher. For example, Decision Tree had a CV mean increase from 0.9418 to 0.9472, but its test F1 decreased from 0.9517 to 0.9231 and its CV standard deviation increased substantially. Hence, dimensionality reduction alone is not sufficient to guarantee improved generalization.

12.4 Linear Models versus Ensemble Models

Logistic Regression was particularly robust: its accuracy remained 0.9737, while F1 changed only from 0.9793 to 0.9790. SVM retained strong validation performance but experienced a test F1 reduction after PCA. Among ensemble models, Gradient Boosting was unchanged in test F1, whereas Random Forest, AdaBoost, and XGBoost generally declined after PCA. This suggests that the original feature representation contains useful structure that nonlinear tree ensembles can exploit directly.

12.5 Stacking Robustness

Stacking achieved a No-PCA test F1-score of 0.9790 and a PCA test F1-score of 0.9650, a decrease of approximately 0.0140. Its CV mean F1 decreased from 0.9807 to 0.9774, while the standard deviation changed only slightly from 0.0089 to 0.0089. Thus, stacking remained relatively stable in terms of fold variability, but its independent test performance was meaningfully reduced by PCA. It can therefore be described as reasonably stable but not fully performance-robust to the dimensionality transformation.

12.6 Best Models

The best No-PCA individual model was SVM, with test F1-score 0.9861 and accuracy 0.9825. The best PCA individual model was Logistic Regression, with test F1-score 0.9790 and accuracy 0.9737. This change in the identity of the strongest classifier shows that dimensionality reduction can alter the relative suitability of different algorithms.

Among the PCA configurations, Logistic Regression also produced an excellent ROC-AUC of 0.9954 and Average Precision of 0.9971. Its confusion matrix contained only three errors out of 114 test samples.

13 Conclusion

The experiment demonstrated the effect of PCA-based dimensionality reduction on a set of machine-learning classifiers using the Breast Cancer Wisconsin Diagnostic dataset. Standardization was applied before PCA, and the number of principal components was selected automatically to retain at least 95% of the training variance. This reduced the feature space from 30 original variables to 10 principal components while retaining 95.2677% of the variance.

The results show that PCA does not automatically improve classification performance. In terms of independent test F1-score, no evaluated model improved after PCA; Gradient Boosting remained unchanged, while the remaining models decreased. Nevertheless, PCA improved cross-validation mean F1 for SVM and Decision Tree and substantially improved Naïve Bayes stability across folds. These observations demonstrate that PCA's usefulness depends on both the data representation and the learning algorithm.

The best No-PCA model was SVM with F1-score 0.9861, whereas the best PCA model was Logistic Regression with F1-score 0.9790. The best PCA Logistic Regression model achieved an ROC-AUC of 0.9954 and Average Precision of 0.9971. Overall, PCA should therefore be regarded as a data-dependent dimensionality-reduction technique rather than a transformation that universally improves predictive performance.

14 References

1. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
2. I. T. Jolliffe and J. Cadima, “Principal component analysis: a review and recent developments,” *Philosophical Transactions of the Royal Society A*, vol. 374, no. 2065, 2016.
3. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
4. L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001.